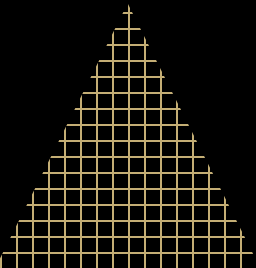


CS-218 Data Structures and Algorithms

LECTURE 21

BY UROOJ AINUDDIN

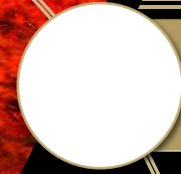




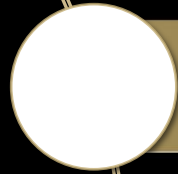
Trees

BOOK 1 CHAPTER 13

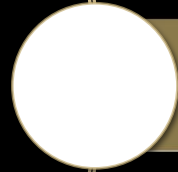




We implemented a class for the binary tree data structure.



We implemented functions to add and remove nodes from a binary tree.



We implemented codes to calculate height, and count nodes and leaves of a binary tree.



We implemented codes to determine the type of a binary tree.



We examined all codes for time and space complexities.

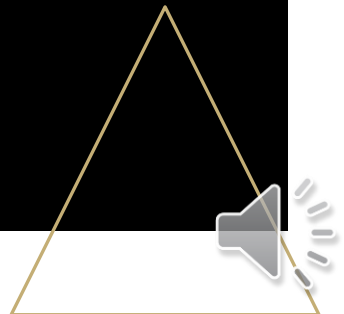
In the last lecture...





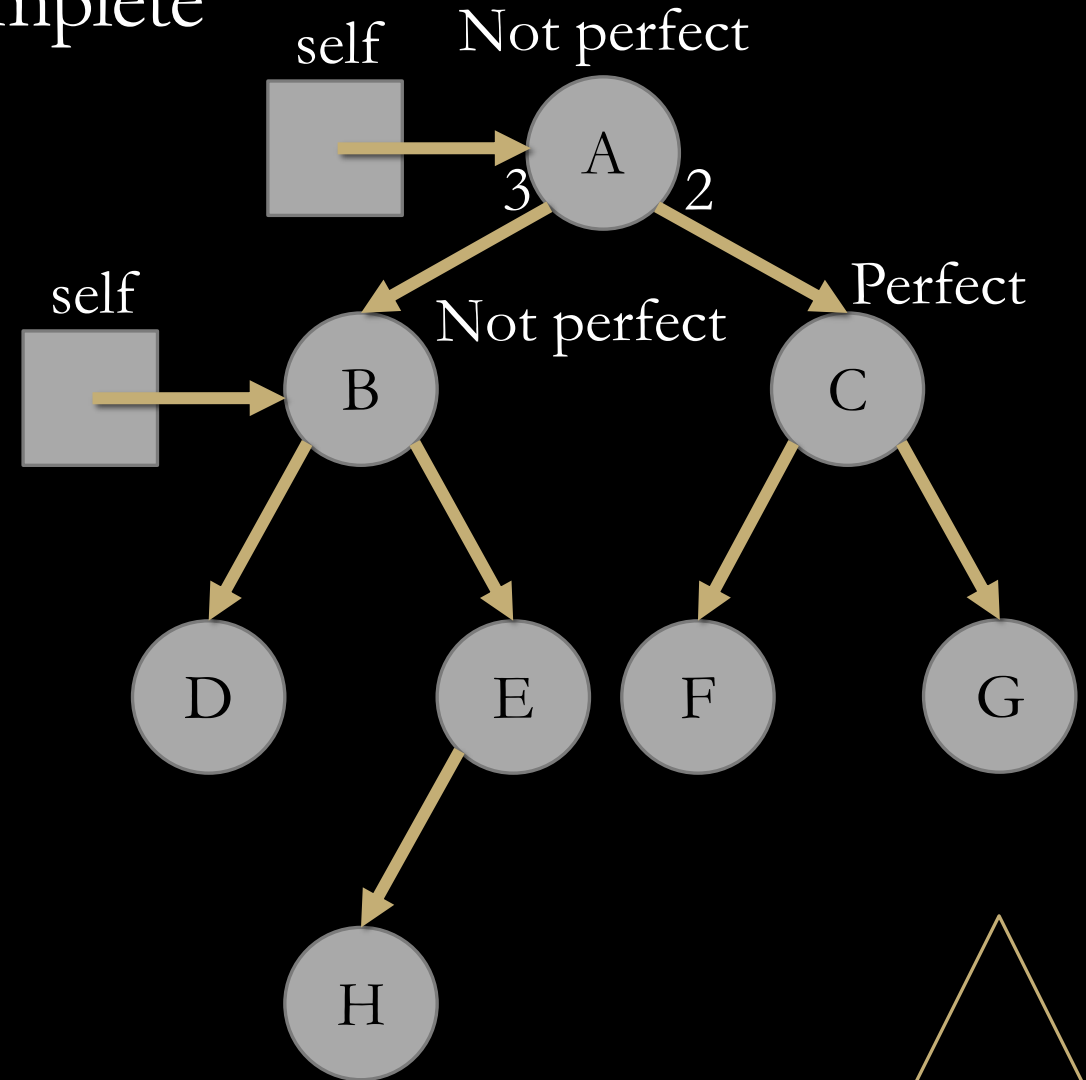
When is a binary tree almost complete?

- If the tree is a PBT, then the tree is an ACBT.
- If the difference in height of the two subtrees of the root is 0, the left subtree is a PBT, and the right subtree is an ACBT, then the tree is an ACBT.
- If the difference in height of the two subtrees of the root is 1, the right subtree is a PBT, and the left subtree is an ACBT, then the tree is an ACBT.



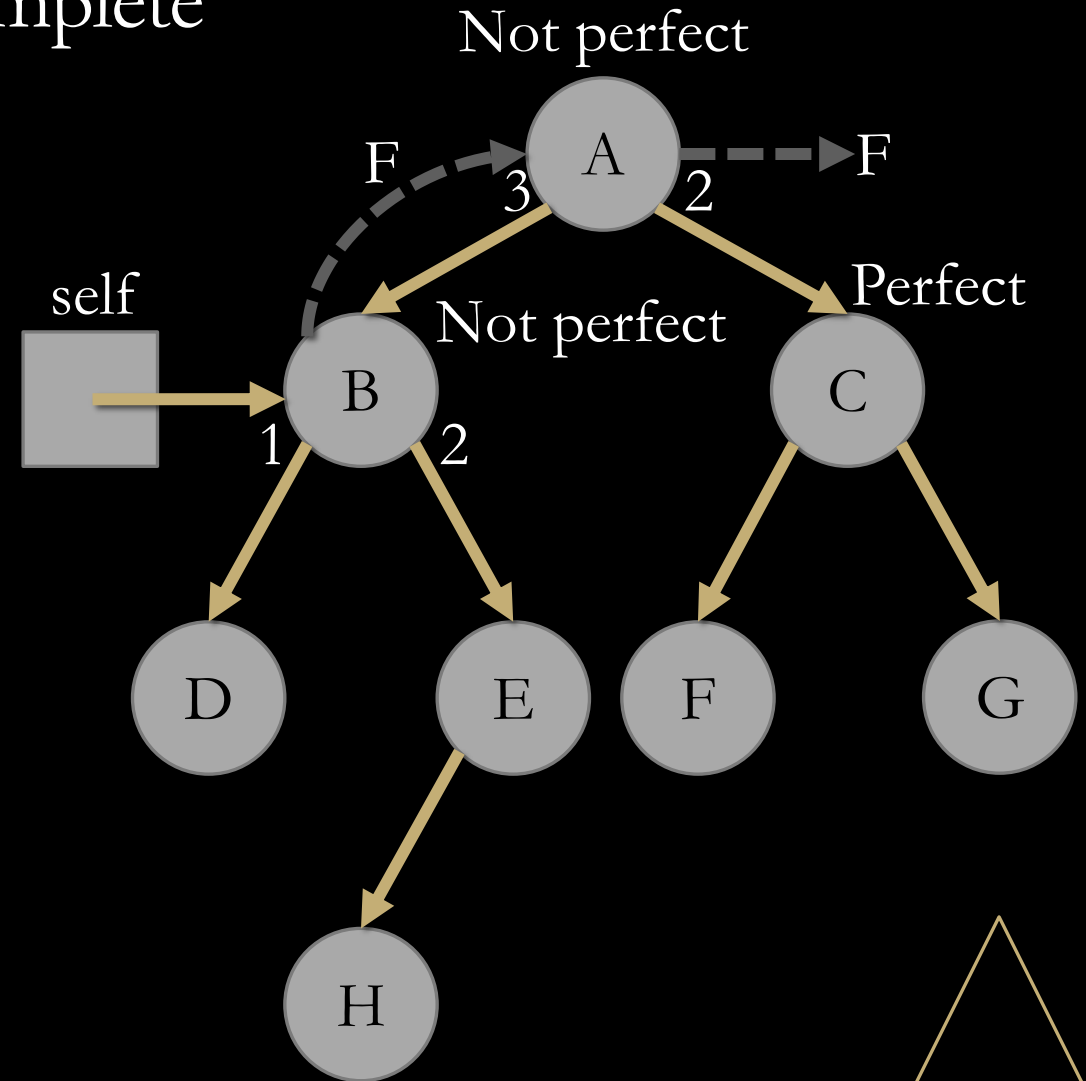
The binarytree.py file - isAlmostComplete

```
def isAlmostComplete(self):  
    if self.isPerfect():  
        return True  
    if self.lc is None:  
        return False  
    hl = self.lc.height()  
    if self.rc is None:  
        if hl == 1:  
            return True  
        return False  
    hr = self.rc.height()  
    if hl == hr and self.lc.isPerfect():  
        return self.rc.isAlmostComplete()  
    if hl == hr + 1 and self.rc.isPerfect():  
        return self.lc.isAlmostComplete()  
    return False
```



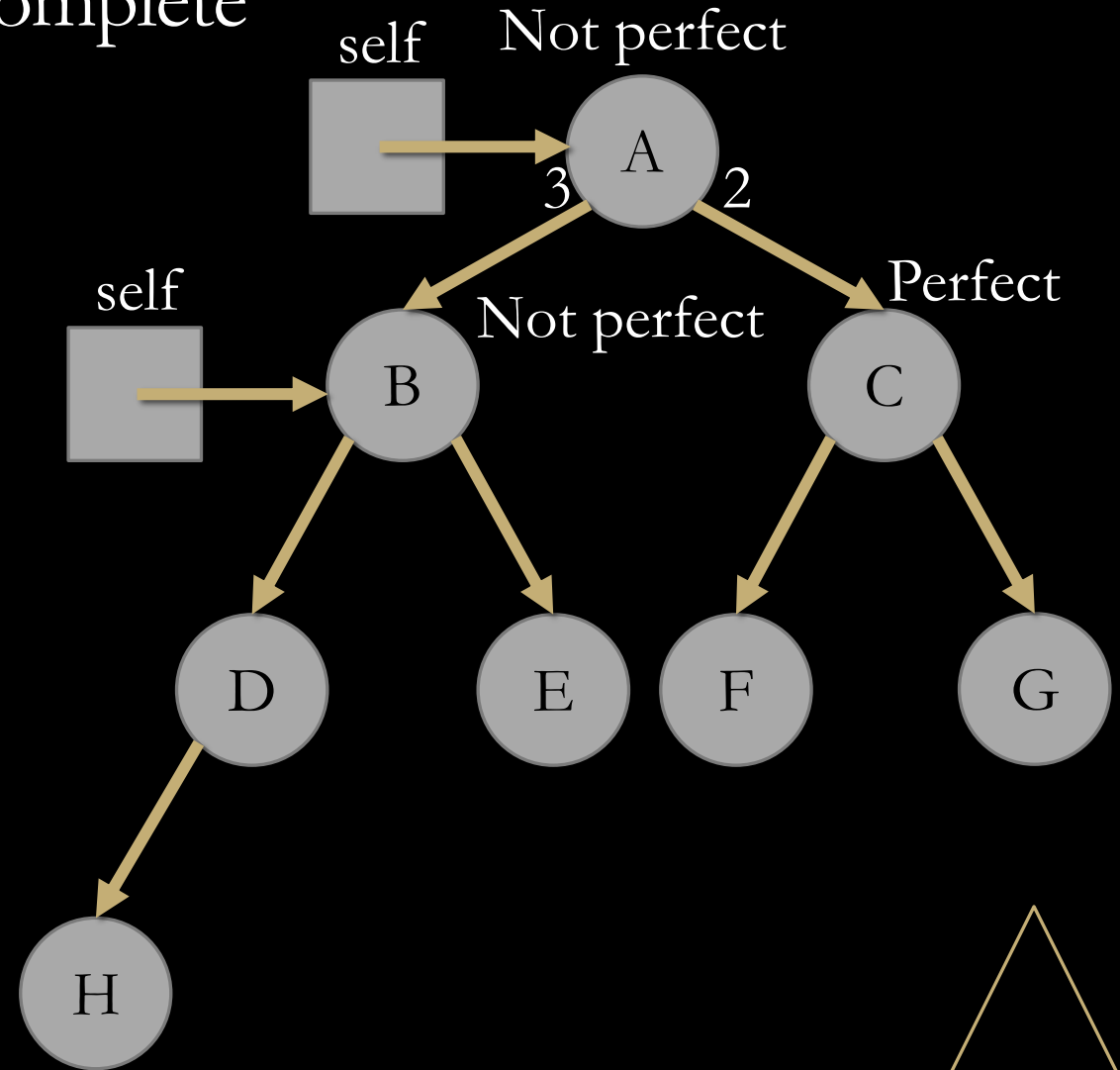
The binarytree.py file - isAlmostComplete

```
def isAlmostComplete(self):  
    if self.isPerfect():  
        return True  
    if self.lc is None:  
        return False  
    hl = self.lc.height()  
    if self.rc is None:  
        if hl == 1:  
            return True  
        return False  
    hr = self.rc.height()  
    if hl == hr and self.lc.isPerfect():  
        return self.rc.isAlmostComplete()  
    if hl == hr + 1 and self.rc.isPerfect():  
        return self.lc.isAlmostComplete()  
    return False
```



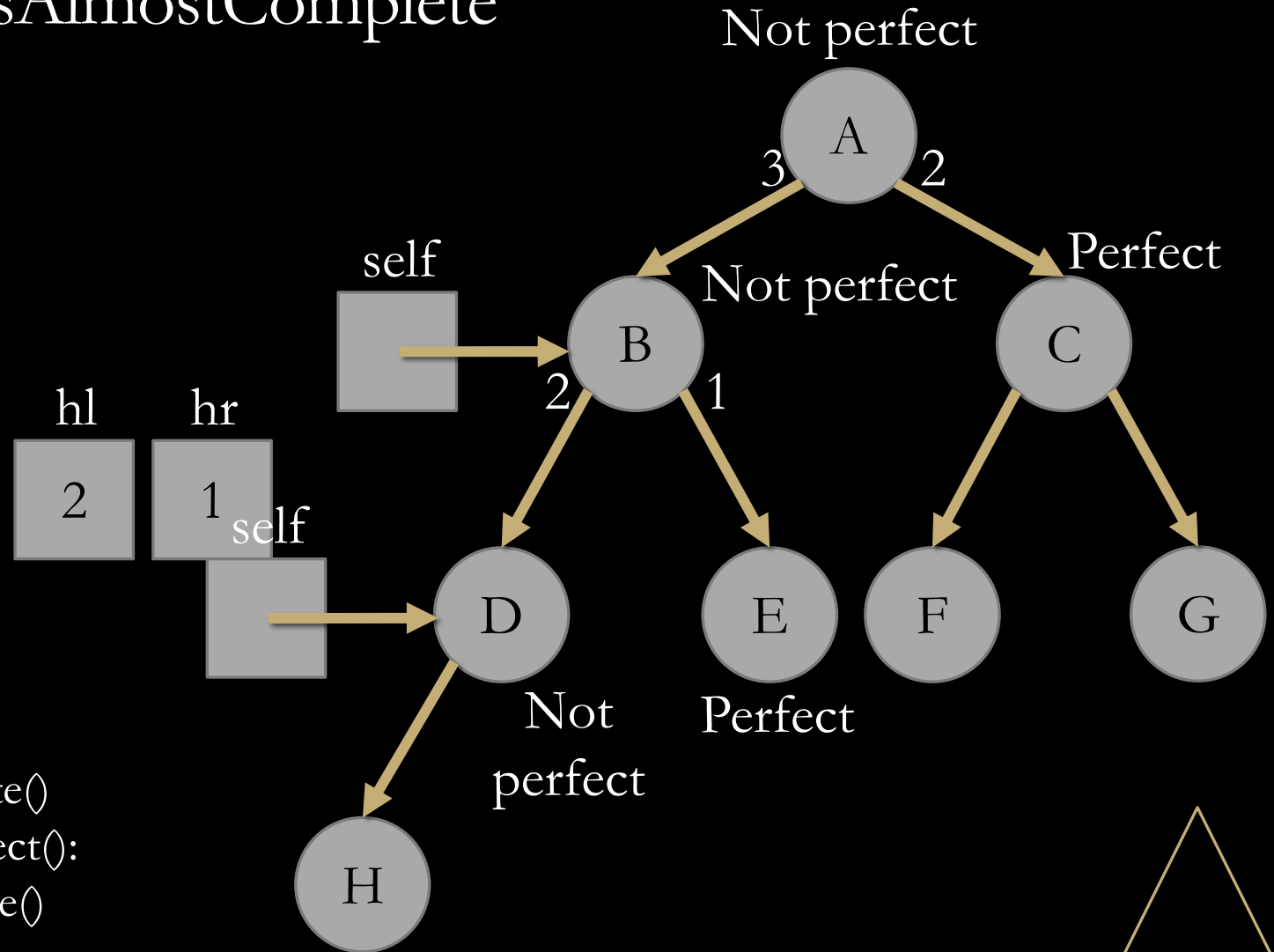
The binarytree.py file - isAlmostComplete

```
def isAlmostComplete(self):
    if self.isPerfect():
        return True
    if self.lc is None:
        return False
    hl = self.lc.height()
    if self.rc is None:
        if hl == 1:
            return True
        return False
    hr = self.rc.height()
    if hl == hr and self.lc.isPerfect():
        return self.rc.isAlmostComplete()
    if hl == hr + 1 and self.rc.isPerfect():
        return self.lc.isAlmostComplete()
    return False
```



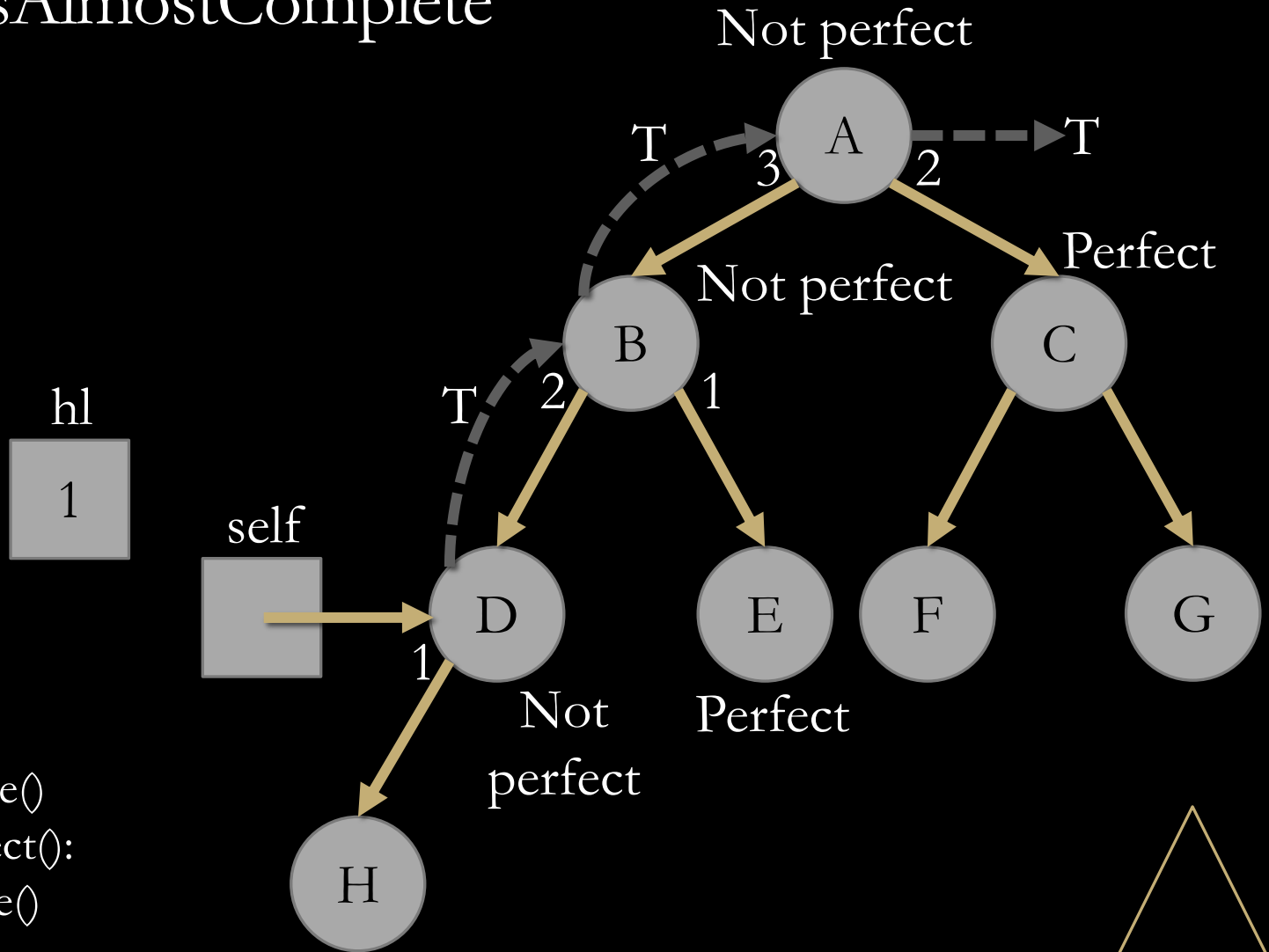
The binarytree.py file - isAlmostComplete

```
def isAlmostComplete(self):
    if self.isPerfect():
        return True
    if self.lc is None:
        return False
    hl = self.lc.height()
    if self.rc is None:
        if hl == 1:
            return True
        return False
    hr = self.rc.height()
    if hl == hr and self.lc.isPerfect():
        return self.rc.isAlmostComplete()
    if hl == hr + 1 and self.rc.isPerfect():
        return self.lc.isAlmostComplete()
    return False
```



The binarytree.py file - isAlmostComplete

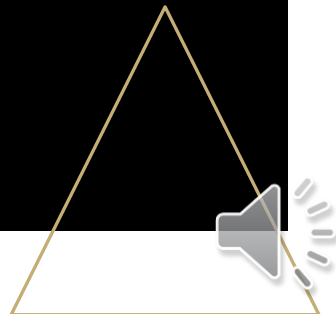
```
def isAlmostComplete(self):  
    if self.isPerfect():  
        return True  
    if self.lc is None:  
        return False  
    hl = self.lc.height()  
    if self.rc is None:  
        if hl == 1:  
            return True  
        return False  
    hr = self.rc.height()  
    if hl == hr and self.lc.isPerfect():  
        return self.rc.isAlmostComplete()  
    if hl == hr + 1 and self.rc.isPerfect():  
        return self.lc.isAlmostComplete()  
    return False
```



The binarytree.py file - isAlmostComplete

```
def isAlmostComplete(self):
    if self.isPerfect():
        return True
    if self.lc is None:
        return False
    hl = self.lc.height()
    if self.rc is None:
        if hl == 1:
            return True
        return False
    hr = self.rc.height()
    if hl == hr and self.lc.isPerfect():
        return self.rc.isAlmostComplete()
    if hl == hr + 1 and self.rc.isPerfect():
        return self.lc.isAlmostComplete()
    return False
```

1. Draw a BT of height 4 in which only one node (the rightmost node in the last level) is absent. Run the code for this tree.
2. Draw a BT of height 4 in which the first three levels are completely filled and only three nodes (the leftmost nodes) are present in the last level. Run the code for this tree.



The binarytree.py file - isAlmostComplete

```
def isAlmostComplete(self):
    if self.isPerfect():
        return True
    if self.lc is None:
        return False
    hl = self.lc.height()
    if self.rc is None:
        if hl == 1:
            return True
        return False
    hr = self.rc.height()
    if hl == hr and self.lc.isPerfect():
        return self.rc.isAlmostComplete()
    if hl == hr + 1 and self.rc.isPerfect():
        return self.lc.isAlmostComplete()
    return False
```

Time: $O(n \log n)$

- Worst case: The maximum number of function instances are called when the tree is an ACBT. If it not ACBT, this may be detected early with fewer function instances.
- If the ACBT is of height h , $h - 1$ function instances are called, one for each level except the last level. So, the number of function instances called is $O(h)$.
- Height of a PBT of n nodes is $\log_2(n + 1)$.
- Height of an ACBT of n nodes is $O(\log n)$.
- Time complexity of `isPerfect()` is $O(n)$.
- Time complexity of `height()` is $O(n)$.
- A single function instance takes $O(n)$ time.



The binarytree.py file - isAlmostComplete

```
def isAlmostComplete(self):
    if self.isPerfect():
        return True
    if self.lc is None:
        return False
    hl = self.lc.height()
    if self.rc is None:
        if hl == 1:
            return True
        return False
    hr = self.rc.height()
    if hl == hr and self.lc.isPerfect():
        return self.rc.isAlmostComplete()
    if hl == hr + 1 and self.rc.isPerfect():
        return self.lc.isAlmostComplete()
    return False
```

Space: $O(\log n)$

- The function uses 2 variables and the run-time stack.
- Worst case: The maximum number of function instances are called when the tree is an ACBT. If it not ACBT, this may be detected early with fewer function instances.
- If the ACBT is of height h , $h - 1$ function instances are called, one for each level except the last level. The number of activation records pushed is $h - 2$. So, the maximum size of the run-time stack is $O(h)$.
- Height of a PBT of n nodes is $\log_2(n + 1)$.
- Height of an ACBT of n nodes is $O(\log n)$.
- A single function instance takes $O(1)$ space.



Tree traversal

- The tree is a non-linear data structure.
- It requires an algorithm to ensure traversal, i.e., visiting every node exactly once.
- We concentrate on traversal algorithms for binary trees.



Depth-first traversal

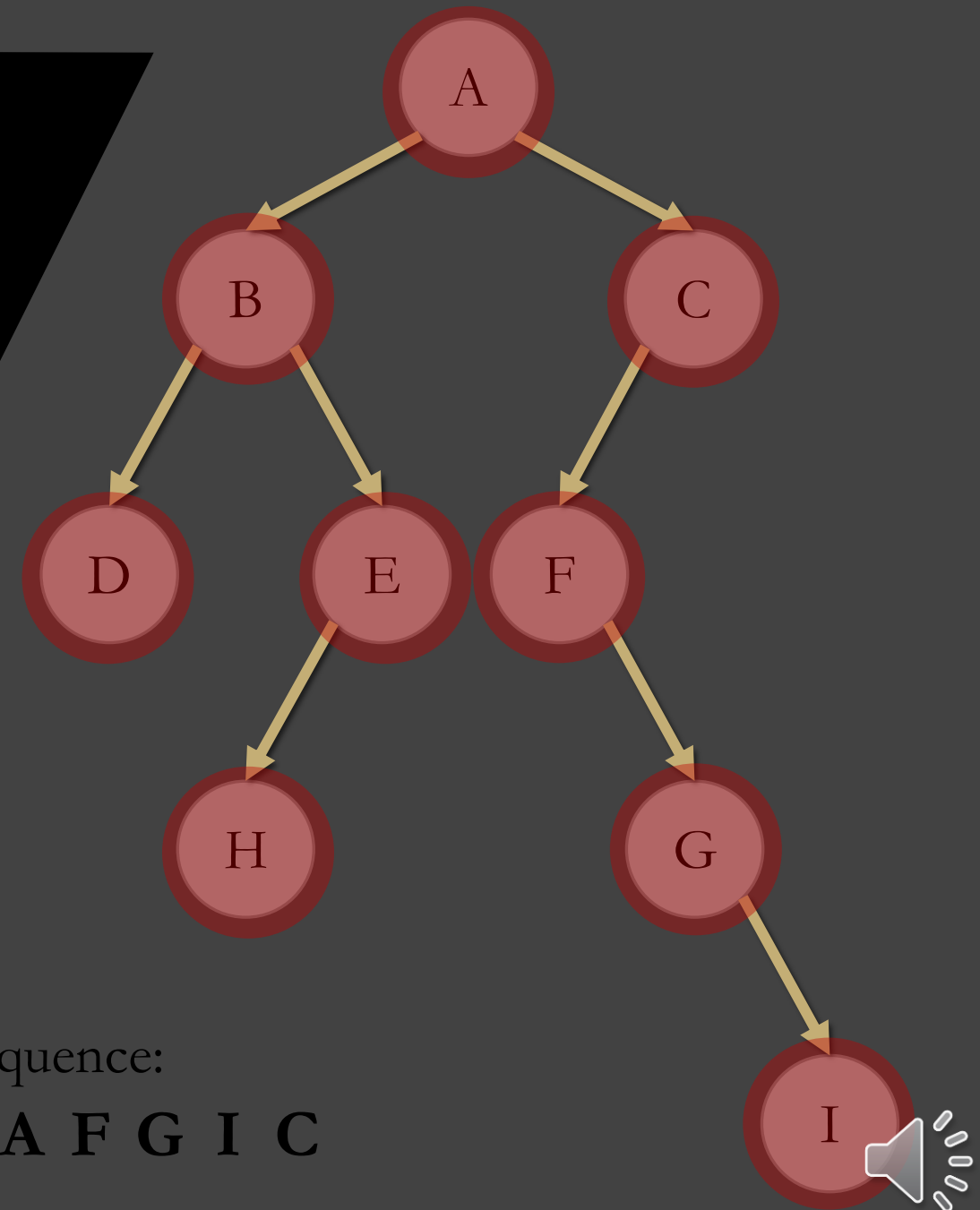
There are three algorithms of this kind:

1. Inorder Traversal
2. Preorder Traversal
3. Postorder Traversal



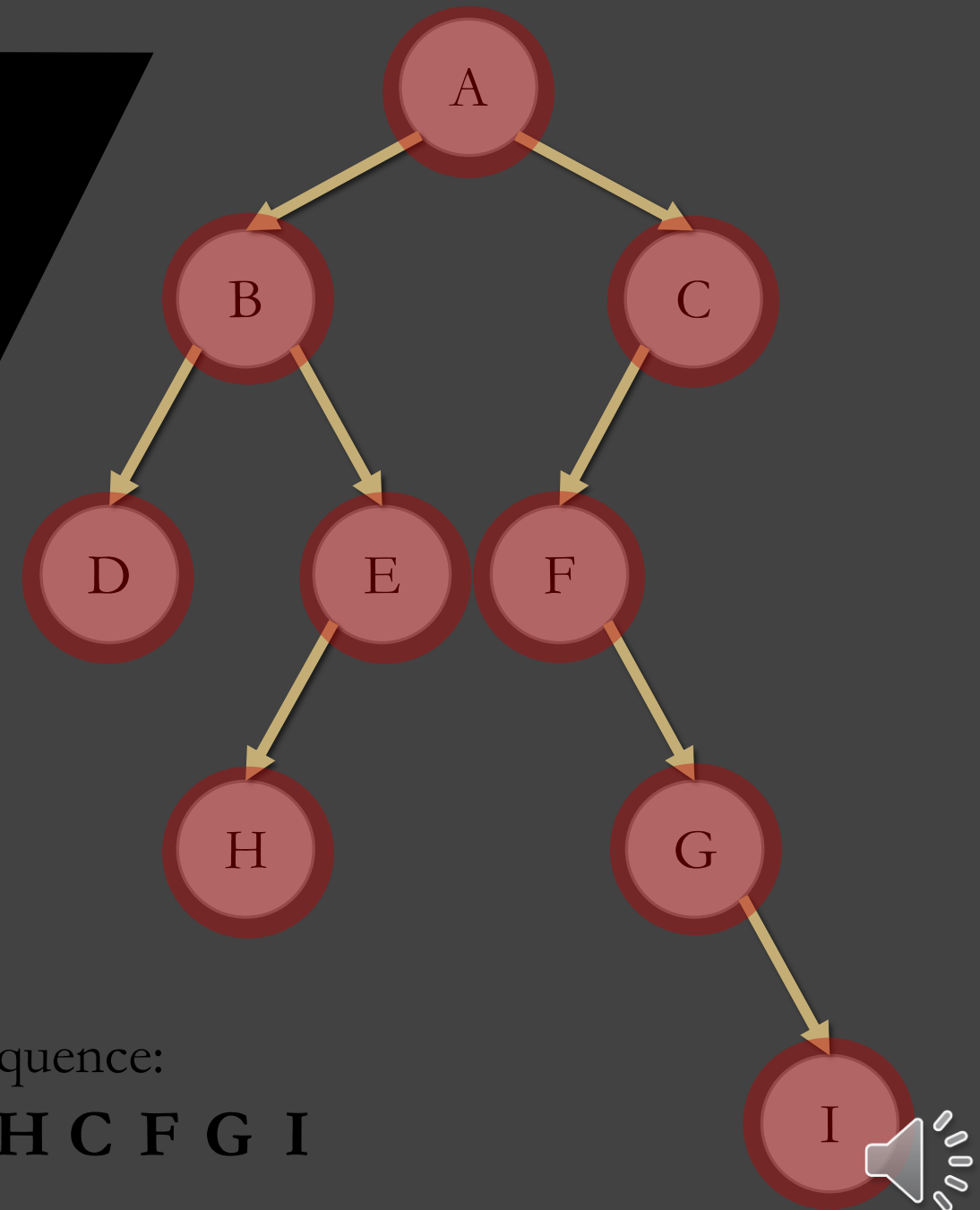
Inorder traversal

- Start at the root. When standing at a node,
 1. Move to the left subtree.
 2. Visit the node.
 3. Move to the right subtree.
- We print the data field of a node when we **visit** it.
- The node is visited **in between** the left and the right child:
LNR.



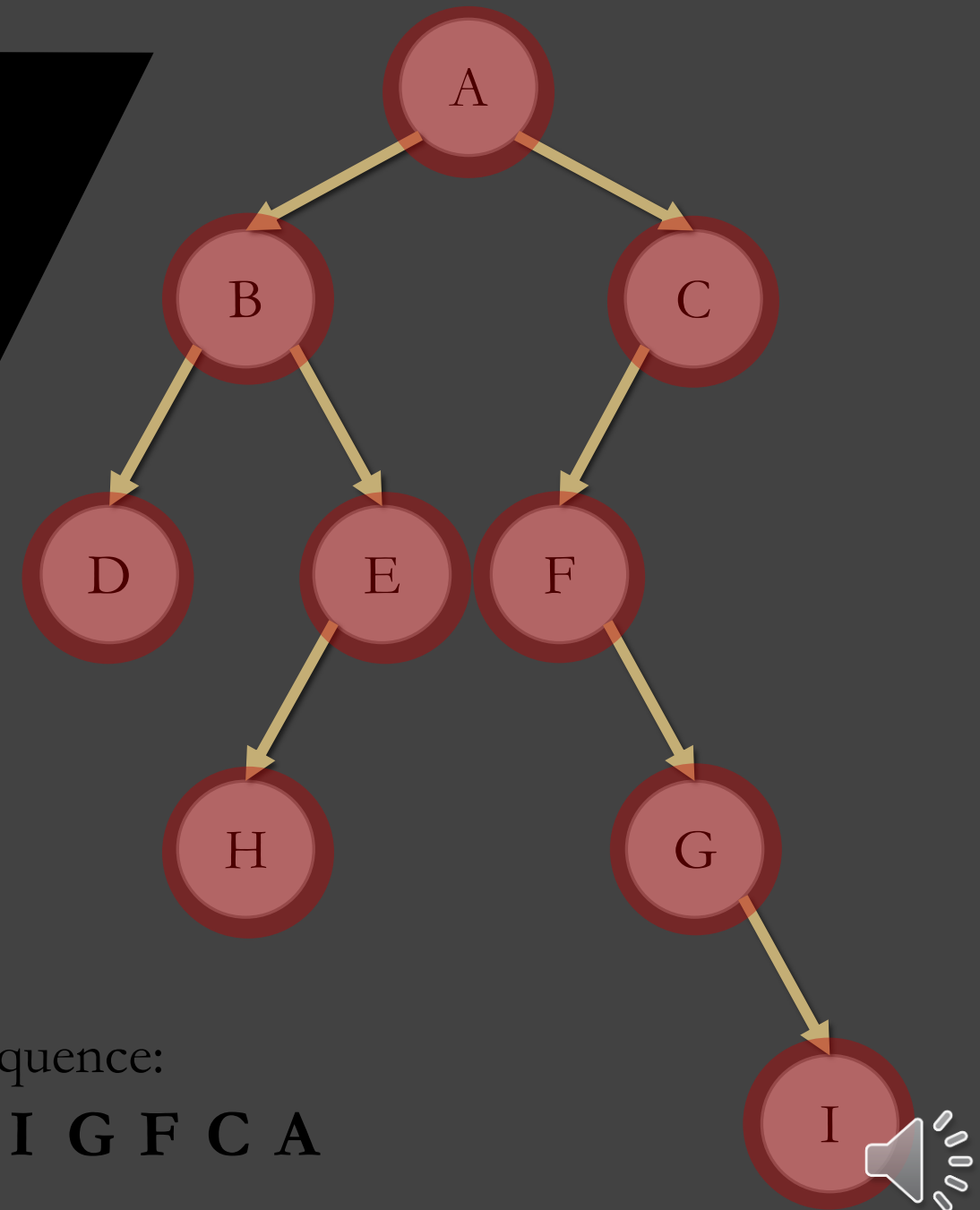
Preorder traversal

- Start at the root. When standing at a node,
 1. Visit the node.
 2. Move to the left subtree.
 3. Move to the right subtree.
- We print the data field of a node when we **visit** it.
- The node is visited **before** the left and the right child: **NLR**.



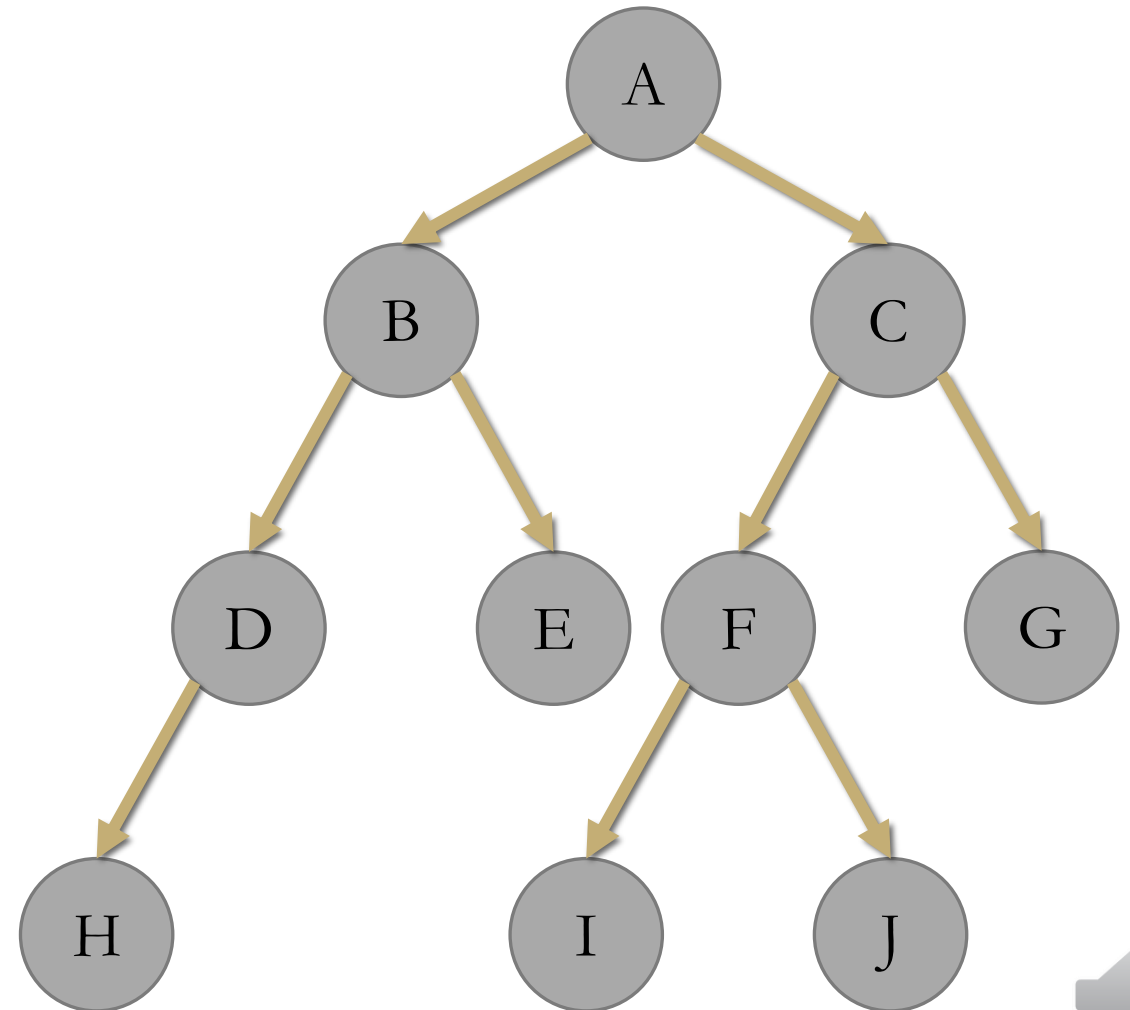
Postorder traversal

- Start at the root. When standing at a node,
 1. Move to the left subtree.
 2. Move to the right subtree.
 3. Visit the node.
- We print the data field of a node when we **visit** it.
- The node is visited **after** the left and the right child: **LRN**.



Homework

Give traversal sequences generated by all 3 algorithms for the tree shown here:



The binarytree.py file - traversein

```
def traversein(self):  
    if self.lc is not None:  
        self.lc.traversein()  
    print(self.data, end=" ")  
    if self.rc is not None:  
        self.rc.traversein()
```

Time: $O(n)$

- The function is called for every node.
- If there are n nodes, the recursion creates n function instances.
- A single function instance takes $O(1)$ time.



The binarytree.py file - traversein

```
def traversein(self):  
    if self.lc is not None:  
        self.lc.traversein()  
    print(self.data, end=" ")  
    if self.rc is not None:  
        self.rc.traversein()
```

Space: $O(h)$

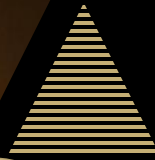
- The maximum number of activation records are pushed in the longest branch of the tree.
- If the height of the tree is h , the number of activation records pushed on to the run-time stack is $h - 1$.
- A single function instance takes $O(1)$ time.



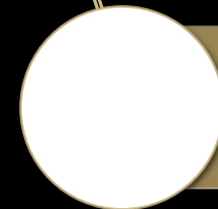
Homework

1. Write code for a function of class Btree called `traversepre(self)` that traverses a binary tree using preorder traversal algorithm.
2. Write code for a function of class Btree called `traversepost(self)` that traverses a binary tree using postorder traversal algorithm.





We implemented code to determine if a binary tree is an ACBT.



We were introduced to depth-first tree traversal algorithms.



We implemented code for inorder traversal.



We examined all codes for time and space complexities.

So, what did we learn today?



Things to do

Read the book!

Email suggestions on content or quality of this lecture at uroojain@neduet.edu.pk

Note your questions and put them up in the relevant online session.

